SOLID Design Principles

Overview

SOLID is a collection of five object-oriented design principles that improve maintainability and extensibility while reducing coupling between software components. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

SOLID is a collection of five object-oriented design principles that improve maintainability and extensibility while reducing coupling between software components. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

SOLID is a collection of five object-oriented design principles that improve maintainability and extensibility while reducing coupling between software components. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

SOLID is a collection of five object-oriented design principles that improve maintainability and extensibility while reducing coupling between software components. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

SOLID is a collection of five object-oriented design principles that improve maintainability and extensibility while reducing coupling between software components. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Single Responsibility Principle

Each class should have one reason to change. Separating responsibilities improves readability, simplifies testing, and reduces unintended side effects. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Each class should have one reason to change. Separating responsibilities improves readability, simplifies testing, and reduces unintended side effects. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Each class should have one reason to change. Separating responsibilities improves readability, simplifies testing, and reduces unintended side effects. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Each class should have one reason to change. Separating responsibilities improves readability, simplifies testing, and reduces unintended side effects. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Each class should have one reason to change. Separating responsibilities improves readability, simplifies testing, and reduces unintended side effects. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Open/Closed Principle

Software should be open for extension but closed for modification. Use inheritance, composition, or interfaces to introduce new behavior without changing stable code. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Software should be open for extension but closed for modification. Use inheritance, composition, or interfaces to introduce new behavior without changing stable code. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Software should be open for extension but closed for modification. Use inheritance, composition, or interfaces to introduce new behavior without changing stable code. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Software should be open for extension but closed for modification. Use inheritance, composition, or interfaces to introduce new behavior without changing stable code. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Software should be open for extension but closed for modification. Use inheritance, composition, or interfaces to introduce new behavior without changing stable code. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Liskov Substitution Principle

Derived classes should be substitutable for their base classes without changing program correctness. Violations often signal improper inheritance relationships. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Derived classes should be substitutable for their base classes without changing program correctness. Violations often signal improper inheritance relationships. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Derived classes should be substitutable for their base classes without changing program correctness. Violations often signal improper inheritance relationships. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Derived classes should be substitutable for their base classes without changing program correctness. Violations often signal improper inheritance relationships. Practical examples, design

considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Derived classes should be substitutable for their base classes without changing program correctness. Violations often signal improper inheritance relationships. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Interface Segregation Principle

Prefer multiple focused interfaces over one large interface. Clients should depend only on methods they actually use, resulting in greater modularity. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Prefer multiple focused interfaces over one large interface. Clients should depend only on methods they actually use, resulting in greater modularity. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Prefer multiple focused interfaces over one large interface. Clients should depend only on methods they actually use, resulting in greater modularity. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Prefer multiple focused interfaces over one large interface. Clients should depend only on methods they actually use, resulting in greater modularity. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Prefer multiple focused interfaces over one large interface. Clients should depend only on methods they actually use, resulting in greater modularity. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Dependency Inversion Principle

Depend on abstractions instead of concrete implementations. Dependency injection frameworks help decouple components and improve testability. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Depend on abstractions instead of concrete implementations. Dependency injection frameworks help decouple components and improve testability. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Depend on abstractions instead of concrete implementations. Dependency injection frameworks help decouple components and improve testability. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting

architectural decisions as systems evolve.

Depend on abstractions instead of concrete implementations. Dependency injection frameworks help decouple components and improve testability. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Depend on abstractions instead of concrete implementations. Dependency injection frameworks help decouple components and improve testability. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Composition Over Inheritance

Favor composition when possible to build flexible systems that avoid deep inheritance hierarchies and encourage reusable behaviors. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Favor composition when possible to build flexible systems that avoid deep inheritance hierarchies and encourage reusable behaviors. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Favor composition when possible to build flexible systems that avoid deep inheritance hierarchies and encourage reusable behaviors. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Favor composition when possible to build flexible systems that avoid deep inheritance hierarchies and encourage reusable behaviors. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Favor composition when possible to build flexible systems that avoid deep inheritance hierarchies and encourage reusable behaviors. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Practical Trade-offs

SOLID should guide design decisions rather than become rigid rules. Over-engineering can introduce unnecessary complexity if abstractions are added prematurely. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

SOLID should guide design decisions rather than become rigid rules. Over-engineering can introduce unnecessary complexity if abstractions are added prematurely. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

SOLID should guide design decisions rather than become rigid rules. Over-engineering can introduce unnecessary complexity if abstractions are added prematurely. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

SOLID should guide design decisions rather than become rigid rules. Over-engineering can introduce unnecessary complexity if abstractions are added prematurely. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

SOLID should guide design decisions rather than become rigid rules. Over-engineering can introduce unnecessary complexity if abstractions are added prematurely. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Real-world Example

A notification service may depend on an abstract MessageSender interface, allowing email, SMS, or push notification implementations to be substituted without changing business logic. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

A notification service may depend on an abstract MessageSender interface, allowing email, SMS, or push notification implementations to be substituted without changing business logic. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

A notification service may depend on an abstract MessageSender interface, allowing email, SMS, or push notification implementations to be substituted without changing business logic. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

A notification service may depend on an abstract MessageSender interface, allowing email, SMS, or push notification implementations to be substituted without changing business logic. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

A notification service may depend on an abstract MessageSender interface, allowing email, SMS, or push notification implementations to be substituted without changing business logic. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Summary

Applying SOLID thoughtfully creates systems that adapt more easily to changing requirements while remaining understandable and maintainable. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Applying SOLID thoughtfully creates systems that adapt more easily to changing requirements while remaining understandable and maintainable. Practical examples, design considerations,

common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Applying SOLID thoughtfully creates systems that adapt more easily to changing requirements while remaining understandable and maintainable. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Applying SOLID thoughtfully creates systems that adapt more easily to changing requirements while remaining understandable and maintainable. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.

Applying SOLID thoughtfully creates systems that adapt more easily to changing requirements while remaining understandable and maintainable. Practical examples, design considerations, common mistakes, and recommendations should be evaluated during development and code reviews. Teams benefit from documenting standards, mentoring new developers, and revisiting architectural decisions as systems evolve.